

# Echo/SKA Scaling Plan

---

We're scaling the Echo model [Sridhar and Johansen, 2026] from 180M through 440M, 880M, 1.5B, and 3B parameters. This document lays out the work in phases, ordered by dependency: each phase produces artifacts the next phase consumes.

The team splits into three tracks. Two are largely independent: **eval harness** and **dataset curation**. These interact with the rest of the team through well-defined interfaces (a JSON eval spec and a pretokenization config, respectively) and can otherwise proceed without close coordination. The third track, **core architecture and training**, covers diagnostics, architecture search, scaling laws, and production scaling. Everyone on this track needs shared context from the literature, which is why Phase 0.5 exists.

The current codebase lives across `echo-ska-440m/` (active development) and `koopman-lm-fast/` (180M training infra). The 440M config exists in `config.py` with the updated policy (fixed  $\eta/\gamma$ , LayerScale [Touvron et al., 2021], short-conv,  $\beta$ -gated causal stats). Before any scaling work begins, the codebase needs consolidation.

## Phase 0 , Experimental setup (~1 week)

---

Everything else depends on this. The goal is a clean, reproducible experimental platform that any team member can clone, install, and run.

### Codebase consolidation

Right now there are configs spread across `echo-ska-440m/config.py`, `koopman-lm-fast/config.py`, and `ska_distill/config.py`, and `train_fast.py` exists in at least two directories. Consolidate into one authoritative system:

- A single config module with factory functions: `config_50m()`, `config_180m()`, `config_440m()`, `config_880m()`, `config_1p5b()`, `config_3b()`. Each function should return a frozen dataclass so configs can't be mutated after creation.
- One `train_fast.py` as the entry point for all scales. Deprecate the others with explicit deletion PRs, don't leave dead code around.
- One installable package containing the SKA and Koopman MLP modules. A single `pip install -e .` should get a new team member from clone to training a 50M smoke run in under 10 minutes.

### Numerical correctness suite

Before anyone trains anything, the following tests must pass:

- Cholesky update correctness in fp64 ( $\leq 10^{-16}$  relative error).
- Decode-vs-prefill parity ( $\leq 10^{-4}$  max absolute difference across all output tokens).
- Newton-Schulz vs. Cholesky equivalence ( $\leq 10^{-3}$  in bf16).

- Spectral norm stability under fp32 and bf16: verify that  $\rho(A_w)$  stays bounded for 500 random inputs.

These already exist as scattered reference tests: `test_backward`, `test_mamba_recurrence`, `test_ska_causal`, `test_ska_decode`, and `test_ska_newton_schulz`. Collect them into a single suite that runs before any training commit.

**Ideal setup.** Use `pytest` with markers (`@pytest.mark.correctness`) so the full suite runs in CI and individual tests can be selected during development. Target: the full suite passes in under 2 minutes on a single GPU.

## Unified eval harness

Build a single evaluation command that takes any checkpoint and produces one JSON output containing:

- Perplexity (wikitext).
- MQAR grid [Arora et al., 2024]: vary sequence length (256, 512, 1K, 2K) and number of key-value pairs (4, 8, 16, 32, 64).
- NIAH accuracy at 1K, 2K, 4K, 8K, and 16K context.
- A RULER subset [Hsieh et al., 2024]: at minimum the multi-key NIAH, variable tracking, and common-word extraction tasks at 4K and 8K.
- A BABILong subset [Kuratov et al., 2024]: QA1 (single supporting fact) and QA2 (two supporting facts) at 4K and 8K.
- The SKA-zeroed delta: PPL with all SKA layers zeroed out minus baseline PPL. A delta near zero means SKA isn’t load-bearing; a delta above 0.5 PPL points is a strong positive signal.

The existing scripts, `evaluate.py`, `evaluate_retrieval.py`, `niah_quick.py`, `benchmark_50m.py`, should be wired into this harness rather than rewritten. The harness must work at any model scale without manual config changes.

**Ideal setup.** One command: `python eval_harness.py -checkpoint path/to/ckpt`. Output: a single JSON file plus a wandb summary. The harness should auto-detect model scale from the checkpoint’s config hash and select appropriate sequence lengths and batch sizes. For MQAR, use the exact setup from Arora et al. [2024] to enable direct comparison with published Mamba-2 [Dao and Gu, 2024] and Transformer baselines.

## Reproducibility infrastructure

- Deterministic seeding: fix all sources of randomness (PyTorch, NumPy, CUDA, dataloader shuffle). Two runs with the same config hash should produce identical loss curves for at least the first 1K steps.
- Config hashes stored in every checkpoint, so you can always trace a checkpoint back to its exact config.
- Wandb run grouping by experiment, e.g., `p2a-rank-180m-{hash[:8]}`.
- The `ska_fast.py` `state_dict` hooks that handle fused-to-split projection conversion need to be verified across all scales.

**End-to-end smoke test.** Pretokenize, train 1K steps, eval PPL + NIAH, checkpoint, reload, decode. This full loop should pass on a single GPU in under 30 minutes using the 50M config. This test runs nightly in CI.

## Phase 0.5 , Literature survey (~1–2 weeks, concurrent with Phase 0)

---

Everyone on the core architecture/training track should contribute to this. The goal is a shared, annotated reading list that the whole team can reference throughout the project, so that design decisions are grounded in what’s already known rather than rediscovered through trial and error.

### Topic areas to cover

Each person takes one or two areas, finds the 3–5 most relevant papers, and writes a short summary (half a page each) of what the paper found and what it implies for our work. The areas:

- **SSM retrieval and the recall gap.** Why do SSMs struggle with in-context retrieval compared to Transformers? What’s known about the relationship between state size and recall capacity? Start with Zoology [Arora et al., 2024] and the “Revisiting Associative Recall” line of work.
- **Hybrid architectures.** How are production models (Jamba, Zamba, Mamba-3, Nemotron) mixing SSM and attention layers? What layer ratios do they use? Does Mamba-1 vs. Mamba-2 vs. Mamba-3 [Lahoti et al., 2026] matter in the hybrid?
- **Training objectives beyond next-token prediction.** Birdie [Blouir et al., 2024] showed that selective copying and infilling objectives help SSM retrieval. What other auxiliary objectives have been tried, and what worked?
- **Context window scheduling.** SkyLadder [Zhu et al., 2025], GrowLength, Dataset Decomposition [Kaddour et al., 2024]. What’s the right curriculum for training on progressively longer contexts? Does this transfer to SSMs, or is it attention-specific?
- **Scaling laws for non-Transformer architectures.** Has anyone fit Chinchilla-style curves for Mamba or linear attention? How do the exponents differ? What does that imply for our compute allocation?
- **Koopman operators and spectral methods in sequence models.** What’s the prior art on using Koopman theory for sequence modeling, outside of Echo? How does the SKA operator relate to kernel ridge regression and DeltaNet [Yang et al., 2024]?
- **Numerical stability at scale.** What’s known about Cholesky conditioning, spectral norm control, and mixed-precision (bf16) stability for matrix-valued recurrences? Mamba-3’s QKNorm and the Newton–Schulz literature are starting points.

**Deliverable.** A shared document (Google Doc or repo markdown) with one section per topic area. Each section: 3–5 annotated citations, a half-page summary of the state of knowledge, and a bullet list of “implications for us” that maps the findings to specific decisions in Phases 1–5. This document gets referenced throughout the project; it’s not a write-once assignment.

## Phase 1 , Diagnostics and instrumentation

---

Once the platform is stable, instrument it. We need to know whether SKA layers are actually contributing to the model’s output during training, or if the model is learning to route around them entirely.

### SKA health metrics

Emit the following every  $N$  steps to wandb (suggested:  $N=100$  for 50M runs,  $N=500$  for 440M+):

- Spectral radius of  $A_w$  per SKA layer per head. If it hovers near zero, the operator hasn’t learned anything meaningful. Healthy values are typically in  $[0.3, 0.95]$ ; values above 1.0 indicate instability.
- Write-gate magnitude: the LayerScale [Touvron et al., 2021] gate (for 440M) or the learned `intra_weight` (for the Qwen retrofit). If these stay near zero throughout training, SKA is effectively dead. Plot on a log scale; look for monotonic growth over the first 10K steps.
- Residual-norm contribution: the norm of SKA’s output to the residual stream relative to the Mamba [Gu and Dao, 2023] layers’ contribution. If SKA output is orders of magnitude smaller, it’s not doing real work. Compute as  $\|x_{\text{SKA}}\| / \|x_{\text{Mamba}}\|$  averaged over a batch.
- The gap diagnostic:  $\|A_w^2 - A_w\|^2 / \|A_w\|^2$ , which indicates whether the squaring bias is dominating the operator. Values above 0.5 suggest the learned  $A_w$  is far from the “ideal” square root.
- $\lambda_{\min}(\tilde{G})$ : monitors Cholesky conditioning. If this drops below the ridge  $\varepsilon$ , the Cholesky is relying entirely on regularization and the operator is effectively rank-deficient.

This logging module needs to be cheap enough to run at the chosen frequency without meaningfully slowing down training. Profile the overhead explicitly: it should be under 3% of step time.

### Load-bearing test

Make the four-mode SKA-zeroing eval (from earlier work) a standard mode in the harness. If PPL barely changes when you zero out SKA, the layers aren’t load-bearing. The four modes are: (1) full model, (2) SKA zeroed, (3) Mamba zeroed, (4) both zeroed. Reporting all four gives a clean decomposition of what each component contributes.

### Gradient-flow tracking

Track gradient flow through SKA layers specifically: the rank of the effective Jacobian and gradient norm ratios compared to the Mamba branches. This tells us whether SKA is receiving useful learning signal or sitting in a flat region of the loss landscape.

**What to watch for:** if the gradient norm through SKA projections is  $10\times+$  smaller than through Mamba’s input projection, the per-group LR ratios (Phase 2) probably need adjustment. The differential-LR setup in `train_ska_deepspeed.py` already has separate groups for SKA projections,  $\gamma/\eta$ , and Koopman eigenvalues, the diagnostics here tell you whether those ratios are actually right.

**A note on training duration.** SKA layers may need very long training before they become load-bearing. There’s some evidence in the Transformer literature that long-context retrieval capabilities emerge late in pretraining, well after short-range language modeling has converged [Chan et al., 2022, Singh et al., 2023]. The diagnostics from this phase will help distinguish “SKA is dead” from “SKA hasn’t kicked in yet.” The Phase 2 sweeps should include runs long enough to test this.

**Relevant prior work.** Blouir et al. [2024] showed that SSM retrieval can be dramatically improved through training procedure alone, without architecture changes, by combining bidirectional input processing with a dynamic mixture of retrieval-focused pretraining objectives (selective copying, infilling) optimized via RL. This suggests that if the Phase 1 diagnostics show SKA is inactive, the first thing to try may be changing the training objective mix, not the architecture. Separately, Mamba-3 [Lahoti et al., 2026] introduced QKNorm (“BCNorm” in SSM terminology) to stabilize training at scale; we should track whether SKA layers exhibit the same instabilities that motivated this, and consider adding a similar norm to the SKA branch if they do.

## Phase 2 , Small-scale architecture and hyperparameter search

---

With diagnostics in place, sweep architecture and hyperparameters at small scale where iteration is cheap. The search is structured in three stages of increasing fidelity.

### Phase 2a: Exploratory sweep (1M scale, thousands of trials)

Full search space with structural variants included. Use Optuna with aggressive pruning, most runs can be killed after a few hundred steps if the diagnostics from Phase 1 show SKA is inactive. Specifically, use Optuna’s median pruner: if a trial’s MQAR accuracy at step 500 is below the median of completed trials, kill it.

The search space should cover:

- SKA rank ( $r$ ). Current configs use  $r = 48$  at 180M ( $d=768$ ),  $r = 64$  at 370M ( $d=1024$ ), and  $r = 96$  at 440M ( $d=1024$ ). Does rank need to scale with `d_model`, and at what ratio? Search over  $r \in \{32, 48, 64, 96, 128\}$ .
- Number of SKA layers and placement strategy. 180M uses 6/24 (25%), 440M uses 8/34 (~24%). The `_evenly_spaced_indices` function biases toward later layers; the original Echo paper [Sridhar and Johansen, 2026] clusters SKA in the middle-to-deep range (layers 8 and 16 out of 24). Search over fraction (15%, 20%, 25%, 33%) and placement (uniform, late-biased, middle-clustered) jointly.
- Chunk size:  $\{32, 64, 96, 128\}$ .
- Per-group learning rates. The differential-LR approach in `train_ska_deepspeed.py` uses SKA projections at  $5 \times 10^{-5}$ ,  $\gamma/\eta$  at  $25 \times$  base, Koopman eigenvalues at  $10 \times$ , norms at  $5 \times 10^{-6}$ , DeltaNet [Yang et al., 2024] at  $1 \times 10^{-5}$ , MLPs at  $1.5 \times 10^{-5}$ . These ratios were hand-tuned at one scale and need validation. Search the ratios on a log scale (e.g.,  $\gamma/\eta$  multiplier  $\in \{5, 10, 25, 50\}$ ).
- Ridge  $\epsilon$ :  $\{10^{-4}, 10^{-3}, 10^{-2}\}$ . At rank 128+,  $10^{-3}$  may not be enough for Cholesky stability.
- Beta init (write gate):  $\sigma^{-1}(\{0.1, 0.3, 0.5\})$ .
- LayerScale [Touvron et al., 2021] init:  $\{10^{-5}, 10^{-4}, 10^{-3}\}$ .
- Short-conv kernel width ( $\{4, 8, 16\}$ ) and gate init ( $\{10^{-3}, 10^{-2}, 10^{-1}\}$ ).
- QKNorm on the SKA branch: Mamba-3 [Lahoti et al., 2026] found that applying RMSNorm to queries and keys (“BCNorm”) was essential for stable training at scale. Test with and without.

In addition to the architecture/HP axes above, the Phase 2a sweep should include at least one arm that tests the Birdie-style training objective mix [Blouir et al., 2024]: next-token prediction + selective copying + infilling, with the mixture weights optimized by the Optuna trial. If training objectives dominate architecture choices in the importance ranking, that changes the entire plan.

Output: axis importance rankings, top configs, falsified hypotheses. Concretely, produce a wandb report showing (a) the Pareto front on MQAR accuracy vs. wikitext PPL, (b) feature importance from Optuna’s fANOVA, and (c) a short list of configs to promote to Phase 2b.

### Phase 2b: Confirmatory sweep (5M/20M scale, hundreds of trials)

Narrow the search around 2a winners. Run the full diagnostic suite on each trial. Check transfer monotonicity: do the rankings from 1M hold at 5M and 20M? If a config that was top-3 at 1M drops below median at 20M, that axis doesn’t transfer and should be fixed rather than swept at larger scale.

## Phase 2c: Validation (50M/180M scale, tens of trials)

Single-axis ablations around the best configs from 2b. Final config selection for the 440M run. Each ablation should include the SKA-zeroed delta to confirm the layers are load-bearing at this scale. Train each run for at least 10B tokens to give SKA time to activate (see the training-duration note in Phase 1).

## Phase 3 , Scaling laws (50M / 180M / 440M)

---

With a validated architecture and HP recipe, fit scaling laws for Echo specifically. The methodology follows [Hoffmann et al. \[2022\]](#), with adaptations for the Echo architecture.

1. Train a grid of Echo models (10M to 200M+) at varying token budgets. Suggested grid: 5 model sizes  $\times$  4–5 token budgets each, for 20–25 runs total. Use the configs validated in Phase 2c. Each run trains to completion (no early stopping) and records final eval loss from the harness.
2. Fit the Chinchilla loss curves:  $L(N, D) = E + A N^{-\alpha} + B D^{-\beta}$ . Use `scipy's curve_fit` with multiple random initializations to avoid local minima. Report 95% confidence intervals on  $\alpha$  and  $\beta$ .
3. Use the fitted curves to predict: given a fixed compute budget (say, 4 B200-days), what's the optimal  $N$  (params) and  $D$  (tokens)?
4. Compare the scaling exponents to published Mamba-2 [\[Dao and Gu, 2024\]](#) and Transformer numbers. SKA carries  $O(r^2)$  state that amortizes differently from attention, so the compute-optimal frontier may shift.

The output is a decision artifact: a recommendation on  $(N, D)$  for the available compute budget, with confidence intervals. This is what gates the commitment of serious GPU time in Phase 5.

**What “ideal” looks like.** The fitted  $\alpha$  for Echo should be close to Chinchilla's  $\alpha \approx 0.34$  for Transformers. If it's significantly lower, Echo gets less marginal benefit from scaling parameters, and we should train smaller models longer. If it's significantly higher, the opposite, and we should scale aggressively. The Epoch AI replication [\[Besiroglu et al., 2024\]](#) found that Chinchilla's third approach had issues with optimizer convergence; use their corrected fitting procedure.

**Starting point.** The CS336 Assignment 4 spec has the full Chinchilla methodology. Use `train_fast.py` as the training harness.

## Phase 4 , Data scale-up

---

### Pretraining mix design

The current data situation has two problems. First, SCROLLS [\[Shaham et al., 2022\]](#) is too short: most examples fall under 2K tokens, and the model starts memorizing after a few epochs. Second, the Qwen retrofit work showed that when training was 90% local-prediction data and 10% retrieval, the Koopman operator never learned long-range retrieval, NIAH accuracy dropped to chance past 1K tokens. The data mix was the bottleneck, not the architecture.

The pretraining mix needs an explicit retrieval fraction and a controlled context-length distribution. Candidates for the retrieval portion:



- SCROLLS subsets (gov\_report, qasper, narrative\_qa, quality, contract\_nli), filtered to examples actually exceeding 2K tokens. Drop summ\_screen\_fd if most of it falls short. Measure the actual distribution of answer distances (distance in tokens from the supporting evidence to the query) per subset, this is more informative than raw sequence length.
- BABILong [Kuratov et al., 2024] and LongBench (already partially integrated). BABILong is particularly useful because it provides reasoning tasks at controlled distances, not just retrieval.
- Multi-doc QA: HotpotQA, MuSiQue, 2WikiMultiHopQA, padded with PG-19 filler to push context to 8K+. The padding strategy matters: insert the filler *between* evidence and query, not at the end, so the model must recall across the gap.
- Synthetic NIAH at scale, extending the 440M pretokenizer with varied key-value formats, multi-hop chains, and different distractor densities. Target at least 5 different key-value formats (named entities, dates, numbers, code variables, structured facts) to prevent the model from overfitting to one retrieval pattern.
- Code: StarCoder data filtered for files over 4K tokens. Code has natural long-range dependencies (function defined early, called much later).
- note: look into the Nvidia Nemotron/megatron papers and repository, they have publicly defined their pretraining datasets. Same for Marin community models.

For the general LM portion: PG-19 (long books, anti-forgetting), RedPajama-v2 or SlimPajama.

Target: 30B+ tokens with at least 30% retrieval-structured data at 4K+ context. The dataset\_weighted.py MemmapPackedDataset already supports per-token recall weights; build on that.

**What “ideal” looks like.** The final data mix should have a clear context-length histogram showing substantial mass at 4K, 8K, and 16K. Plot the answer-distance distribution separately: at least 20% of retrieval examples should have answer distances above 2K tokens, otherwise SKA has no reason to activate. The Qwen retrofit failure happened because this number was near zero.

## Long-context retrieval fine-tuning

Apply a masked-loss retrieval fine-tuning recipe to a Phase 3 checkpoint. The key open questions:

- What to mask: context only, or context and query? Answer-only masking gives maximum retrieval pressure but discards language modeling signal.
- Weighting vs. masking: the 440M pretokenizer uses recall\_weight=4 on answer spans; the deepspeed scripts use 5–10×. Is multiplicative upweighting better than hard masking? Run a direct comparison at 50M scale with the Phase 1 diagnostics tracking SKA activation under both regimes.
- Anti-forgetting mix: the retrofit work tried 70% retrieval / 30% general. Start there and adjust based on the PPL/NIAH tradeoff curve.
- BPE boundary handling: context–query separation must be tokenized separately to avoid cross-boundary merges that shift the mask.

Validate that the recipe doesn’t regress PPL beyond an agreed threshold (set before training begins, not after). A reasonable starting point: PPL regression  $\leq 0.3$  points on wikitext in exchange for  $\geq 20\%$  absolute improvement on NIAH at 4K.

## Phase 5 , Production scaling (440M → 880M → 1.5B → 3B)

---

Each scale-up step is gated by explicit go/no-go criteria from the eval harness. 440M is the first “real” run: it validates that the Phase 2–3 findings transfer past 180M. Each subsequent step is a separate decision, not a foregone conclusion.

Go/no-go criteria (proposed):

- SKA-zeroed PPL delta exceeds 0.5 points (SKA is load-bearing at this scale).
- NIAH accuracy at 4K+ context  $\geq 80\%$  (the target set in Phase 4 may revise this).
- PPL is within 0.3 points of a size-matched Mamba-3 [Lahoti et al., 2026] or Mamba-2 [Dao and Gu, 2024] baseline (we aren’t paying an architecture tax for the SKA layers).
- No numerical instability:  $\lambda_{\min}(\tilde{G})$  stays above  $\varepsilon/2$ , spectral radius  $\rho(A_w) < 1.0$  for all heads, and gradient norms don’t spike above  $10\times$  the running mean.

Sequence length scheduling deserves attention here. The 440M plan suggests a curriculum (first 30% of training at 2K, next 40% at 4K, final 30% at 8K+), but this hasn’t been validated. Recent work on context window scheduling provides useful guidance. SkyLadder [Zhu et al., 2025] found that starting from very short contexts (even 8 tokens) and linearly increasing to the target length, with roughly 60% of total tokens allocated to the expansion phase, outperforms fixed long-context training on both short- and long-context benchmarks. Kaddour et al. [2024] found that a cyclic schedule (repeatedly growing and resetting the context window) further stabilizes training by reducing gradient variance from long sequences. The Phase 1 diagnostics should be tracked throughout to catch the point at which SKA activates, if it activates late, the curriculum may need to be adjusted accordingly.

**Compute planning.** Use the scaling law fit from Phase 3 to set the token budget at each scale. If the fit says 440M is compute-optimal at 20B tokens on our budget, don’t train it for 50B “just to be safe”, that’s wasted compute that could go toward the 880M run. Conversely, if the fit says we’re undertrained, extend the run before scaling up.

## Phase 6 , In-depth evaluation of the largest committed model

---

Once the largest model that passes go/no-go is trained, evaluate it thoroughly:

- Full RULER [Hsieh et al., 2024] benchmark (not just the subset from the harness): all 13 tasks at context lengths 4K through 32K.
- Full BABILong [Kuratov et al., 2024]: all 20 reasoning tasks at 4K, 8K, 16K, and 32K.
- Agentic tool-trace benchmarks, if applicable at this scale.
- Comparison to size-matched Mamba-3 [Lahoti et al., 2026] and hybrid Transformer baselines. Mamba-3 is the right SSM baseline now: it includes QKNorm for stability, a refined SSD layer, and published retrieval numbers on NIAH and MQAR. Train the baseline on the same data mix for a fair comparison; don’t compare against checkpoints trained on different data.
- Targeted failure analysis: where does SKA help, where doesn’t it, and what are the failure modes? In particular, check whether SKA helps more on multi-hop retrieval (BABILong QA2+) than on single-fact retrieval (NIAH), as the Koopman operator’s spectral structure should theoretically enable compositional recall.

## Independent tracks



---

These two tracks run alongside the phased work but don't require tight coordination with the core architecture/training group. Each communicates through a well-defined interface.

**Eval harness** (1–2 people). Built in Phase 0, maintained throughout. The interface is simple: the harness takes a checkpoint path and produces a JSON. The eval team owns the benchmark selection, the scoring logic, the go/no-go dashboards for Phase 5, and scaling the harness to larger models (adjusting batch sizes, sequence lengths, avoiding OOM). They don't need to understand the SKA operator internals, they just need the checkpoint format spec and the config system. As new benchmarks appear (or existing ones get updated), the eval team adds them without blocking anyone else.

**Dataset curation** (1–2 people). The interface is a pretokenization config that produces a `MemmapPackedDataset` with per-token recall weights. The dataset team owns the data mix, the context-length distribution, the answer-distance analysis, and the pretokenizer extensions. They need to understand what SKA needs to activate (long-range retrieval signal, answer distances above 2K tokens), but they don't need to touch training code. Their deliverable feeds directly into Phase 4; the core group consumes the data without modifying the pipeline. Keep a living document of per-dataset statistics (context length distribution, answer distance distribution, number of examples, epochs consumed).

## Core group support tasks

---

These run continuously within the core architecture/training group and require close coordination with the phased work.

**Fast inference and kernel work** (1–2 people). Get the full recurrent inference path (prefill + per-token decode) working correctly and fast:

- Wire the  $O(r^2)$  Cholesky update into the decode step. Currently `_ska_step` re-Choleskys  $G$  every token ( $O(r^3)$ ); the `cholesky_update.update_L_only` function exists but isn't connected. The comment in `_ska_step` explicitly says “carry  $L$  + rank-1 cholupdate for  $O(r^2)$ .” This just needs to be done.
- Benchmark prefill + decode latency at each scale and profile where time is actually spent. Report tokens/sec for prefill (batch) and decode (single-sequence) separately, since they have very different profiles.
- Verify decode-vs-prefix parity at rank 96 and 128.
- Evaluate the Newton–Schulz path (`ska_exact_intrachunk`): an all-matmul alternative to Cholesky for training. Newton–Schulz iterations are pure matrix multiplications, which means they compile well with `torch.compile` and avoid the sequential dependency structure of Cholesky. The Muon optimizer [Jordan et al., 2024] uses 5 Newton–Schulz iterations for approximate matrix orthogonalization and finds this sufficient; profile whether a similar iteration count works for the SKA inverse on B200. If it's competitive, it simplifies the backward and unifies the forward/backward into a single matmul-only codepath.

**Diagnostic tooling** (shared responsibility). The instrumentation from Phase 1, extended as needed. New diagnostics will surface as we encounter unexpected behavior at larger scales.

**Literature survey maintenance** (shared responsibility). The reading list from Phase 0.5 should be updated as new papers come out or as the team encounters relevant work during experiments.

Anyone who finds a paper that changes a design decision should add it with a note on what it implies.

## References

---

- S. Arora, S. Eyuboglu, A. Timalsina, I. Johnson, M. Poli, J. Zou, A. Rudra, and C. Ré. Zoology: Measuring and improving recall in efficient language models. In *ICLR*, 2024.
- T. Besiroglu, E. Erdil, M. Barnett, and J. You. Chinchilla scaling: A replication attempt. *arXiv:2404.10102*, 2024.
- S. Blouir, J. T. H. Smith, A. Anastasopoulos, and A. Shehu. Birdie: Advancing state space models with reward-driven objectives and curricula. In *EMNLP*, 2024.
- S. C. Y. Chan, A. Santoro, A. K. Lampinen, J. X. Wang, A. K. Singh, P. H. Richemond, J. L. McClelland, and F. Hill. Data distributional properties drive emergent in-context learning in transformers. In *NeurIPS*, 2022.
- T. Dao and A. Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *ICML*, 2024.
- A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv:2312.00752*, 2023.
- J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre. Training compute-optimal large language models. In *NeurIPS*, 2022.
- K. Jordan, Y. Li, and E. Meijer. Muon: An optimizer for hidden layers in neural networks. *arXiv:2412.13148*, 2024.
- J. Kaddour, K. Zhu, Y. Lu, L. Liu, and M. Kusner. Dataset decomposition: Faster LLM training with variable sequence length curriculum. In *NeurIPS*, 2024.
- C.-P. Hsieh, S. Sun, S. Krizan, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg. RULER: What’s the real context size of your long-context language models? *arXiv:2404.06654*, 2024.
- Y. Kuratov, A. Bulatov, P. Anokhin, I. Rodkin, D. Sorokin, A. Sorokin, and M. Burtsev. BABILong: Testing the limits of LLMs with long context reasoning-in-a-haystack. In *NeurIPS*, 2024.
- A. Lahoti, K. Y. Li, B. Chen, C. Wang, A. Bick, J. Z. Kolter, T. Dao, and A. Gu. Mamba-3: Improved sequence modeling using state space principles. In *ICLR*, 2026.
- U. Shaham, E. Segal, M. Ivgi, A. Efrat, O. Yoran, A. Haviv, A. Gupta, W. Xiong, M. Geva, J. Berant, and O. Levy. SCROLLS: Standardized comparison over long language sequences. In *EMNLP*, 2022.
- A. K. Singh, S. C. Y. Chan, T. Moskovitz, E. Grant, A. Saxe, and F. Hill. The transient nature of emergent in-context learning in transformers. In *NeurIPS*, 2023.
- A. Sridhar and A. Johansen. Echo: KV-cache-free associative recall with spectral Koopman operators. *arXiv:2605.06997*, 2026.
- H. Touvron, M. Cord, A. Sablayrolles, G. Synnaeve, and H. Jégou. Going deeper with image transformers. In *ICCV*, 2021.

- S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim. Parallelizing linear transformers with the delta rule over sequence length. *arXiv:2406.06484*, 2024.
- T. Zhu, Q. Liu, H. Wang, S. Chen, X. Gu, T. Pang, and M.-Y. Kan. SkyLadder: Better and faster pretraining via context window scheduling. In *NeurIPS*, 2025.